Седловые методы для Traffic Assigment

Патрикеева Елизавета Владимировна*

patrikeeva.ev@phystech.edu

Дмитриев Артем Андреевич†

dmitriev.artem@phystech.edu

Ярмошик Демьян Валерьевич‡

yarmoshik.dv@phystech.edu

Кафедра дискретной математики

Московский физико-технический институт

10 декабря 2025 г.

Аннотация

Аннотация. В работе рассматривается статическая задача распределения транспортных потоков (Traffic Assignment Problem) в постановке пользовательского равновесия Уордропа. Показано её представление в виде выпуклой задачи оптимизации по формуле Бекмана и в седловой форме через лагранжиан с узловыми потенциалами. Обсуждаются условия существования и единственности решения, а также критерии остановки, основанные на UE-gap и duality gap. На этой теоретической базе реализованы два классических алгоритма назначения потоков: алгоритм Франка–Вульфа и метод последовательных усреднений (MSA). Оба метода построены как итеративные схемы «поиск кратчайших путей + обновление потоков» с использованием алгоритмов кратчайших путей и средств векторизации (JAX). На примере тестовой сети Sioux Falls проведено численное сравнение поведения Франка–Вульфа и MSA по скорости сходимости, числу итераций и вычислительным затратам. Результаты вычислительных экспериментов демонстрируют применимость обоих подходов для поиска пользовательского равновесия и выявляют их особенности с точки зрения практической реализации.

Ключевые слова: распределение транспортных потоков; пользовательское равновесие; функция Бекмана; алгоритм Франка–Вульфа; метод последовательных усреднений; сеть Sioux Falls.

*Студентка 2 курса, ФПМИ, МФТИ

†Студент 2 курса, ФПМИ, МФТИ

‡Научный руководитель, к.ф.-м.н., асс

Аннотация

Abstract. This paper deals with the static Traffic Assignment Problem in the framework of Wardrop's User Equilibrium. The problem is formulated as a convex optimization problem via the Beckmann objective and in a saddle-point form using a Lagrangian with node potentials. We discuss conditions of existence and uniqueness of the equilibrium solution as well as stopping criteria based on the UE-gap and the duality gap. On this theoretical basis two classical traffic assignment algorithms are implemented: the Frank–Wolfe algorithm and the Method of Successive Averages (MSA). Both methods are realized as iterative "shortest-path search + flow update" schemes, relying on shortest-path algorithms and vectorized computation (JAX). A numerical comparison of Frank–Wolfe and MSA is carried out on the benchmark Sioux Falls network in terms of convergence speed, number of iterations, and computational effort. The experiments confirm the applicability of both approaches for computing user equilibrium and highlight their differences from the implementation viewpoint.

Keywords: traffic assignment problem; user equilibrium; Beckmann function; Frank–Wolfe algorithm; method of successive averages; Sioux Falls network.

Работа выполнена в рамках учебного проекта по курсу «Математический практикум» в течение осеннего семестра 2025 учебного года.

Вклад авторов:

Патрикеева Е.В. — постановка задачи, разработка алгоритма, написание текста.

Дмитриев А.А. — проведение экспериментов, анализ результатов, визуализация данных.

Научный руководитель Ярмошик Д.В. — общее руководство, консультации, рецензирование текста.

Содержание

1 Введение **3**

1.1 Traffic Assignment Problem (TAP). Формулировка задачи . . . 3

1.2 Равновесие Уордапа (Wardrop Equilibrium) . . . 3

1.2.1 User Equilibrium (UE) . . . 3

1.2.2 System Optimum (SO) . . . 4

1.2.3 Выбор принципа в исследовании . . . 4

1.3 Модель транспортной сети . . . 4

1.4 Седловой подход (Primal–Dual) . . . 6

2 Основная часть **7**

2.1 Формальная постановка Traffic Assignment Problem . . . 7

2.2 Предпосылки существования и единственности решения . . . 7

2.3 Седловая формулировка через лагранжиан ($\min_x \max_\lambda L(x,\lambda)$) . . . 8

2.3.1 Вектор λ^j и матрица Λ_d . . . 8

2.3.2 Матричное уравнение баланса . . . 9

2.3.3 Связь с матрицей спроса D . . . 9

2.3.4 Агрегация по назначениям . . . 11

2.3.5 Построение лагранжиана . . . 12

2.3.6 Явный вид лагранжиана в векторной форме . . . 13

2.3.7 Интерпретация двойственных переменных. . . . 14

2.4 Праймал–дуальная итеративная схема решения . . . 15

2.4.1 Алгоритм . . . 15

2.5 Критерии остановки алгоритма . . . 17

2.5.1 UE-gap (разрыв равновесия) . . . 17

2.5.2 Седловой разрыв (duality gap) . . . 17

2.5.3 Нарушение ограничений . . . 17

2.5.4 Относительные изменения между итерациями . . . 18

2.5.5 Ограничение по числу итераций . . . 18

2.6 Алгоритмы Беллмана–Форда и Дейкстры для кратчайших путей . . . 18

2.6.1 Алгоритм Беллмана–Форда . . . 18

2.6.2 Теорема Беллмана о корректности алгоритма . . . 20

2.6.3 Теорема о детектировании отрицательных циклов . . . 21

2.6.4 Алгоритм Дейкстры . . . 21

2.6.5 Теорема Дейкстры (1959) . . . 22

2.6.6 Восстановление кратчайших маршрутов . . . 24

2.6.7 Batch-обработка множества O–D пар . . . 24

2.7 Алгоритм Джонсона (1977) . . . 25

2.8 Алгоритм Франка–Вульфа. . . . 27

2.9 Метод последовательных усреднений (MSA). . . . 27

2.10 Условия корректности и сходимости метода 28

3 Описание практической работы **30**

1 Введение

1.1 Traffic Assignment Problem (TAP). Формулировка задачи

Определение 1 (Traffic Assignment Problem (TAP))**.** Задача о том, как распределить заданный транспортный спрос (сколько людей или машин едет из точки A в точку B) по дорогам сети так, чтобы система была устойчивой.

Время проезда по дороге растёт, если по ней едет больше машин. Мы ищем такое распределение потоков, при котором никто не может улучшить своё время, выбрав другой маршрут.

Объекты исследования:

- Сеть (дороги, перекрёстки)
- OD-матрица (спрос между парами узлов)
- Функции затрат (время зависит от потока)

Задача

- Найти равновесие, где «все довольны»

1.2 Равновесие Уордапа (Wardrop Equilibrium)

1.2.1 User Equilibrium (UE)

Формулировка:

1. На всех маршрутах, реально используемых между одной и той же OD-парой (o, d), время одинаково и минимально.
2. Ни один водитель не может уменьшить своё время в пути, сменив маршрут.

Математическая запись:

$$t_p(o,d) = t_{p'}(o,d) = \min_{q \in P_{o,d}} t_q(o,d), \quad \forall p, p' \in P^*_{o,d} \tag{1}$$

где:

- $t_p(o,d)$ — время в пути по маршруту p между OD-парой (o,d)
- $P_{o,d}$ — множество всех возможных маршрутов между парой (o,d)
- $P^*_{o,d} \subseteq P_{o,d}$ — множество маршрутов, реально используемых для пары (o,d)

1.2.2 System Optimum (SO)

Формулировка: Потоки распределены так, чтобы минимизировать общее время всех поездок, даже если некоторым водителям это невыгодно индивидуально.
Математическая запись:

$$\min_x \sum_{e \in E} x_e \cdot t_e(x_e) \tag{2}$$

где:

- x_e — поток по ребру (дороге) e
- $t_e(x_e)$ — функция задержки на ребре e (обычно возрастающая функция потока)
- E — множество всех рёбер транспортной сети

1.2.3 Выбор принципа в исследовании

В рамках данного проекта мы рассматриваем принцип **User Equilibrium (UE)**, так как он лучше соответствует реальному поведению водителей, каждый из которых действует рационально, стремясь минимизировать своё собственное время в пути.

1.3 Модель транспортной сети

Структура сети

- **Граф сети:** $G = (V, E)$
  - V — множество узлов (перекрёстков, транспортных развязок)
  - E — множество ориентированных рёбер (дорожных сегментов)
  - $In(V)$ — множество дуг, *входящих* в вершину V.
  - $Out(V)$ — множество дуг, *выходящих* из вершины V.
  - d_{ij} — спрос (количество транспорта), который должен ехать из узла i в узел j.
  - $f_e^{(i,j)}$ поток по дуге e, относящийся к OD-паре (i, j).

Определение потоков Баланс потока для каждой OD-пары:

$$\forall V, (i,j): \quad \sum_{e \in In(V)} f_e^{(i,j)} - \sum_{e \in Out(V)} f_e^{(i,j)} = \begin{cases} -d_{ij}, & \text{если } V = i, \\ d_{ij}, & \text{если } V = j, \\ 0, & \text{иначе.} \end{cases} \tag{3}$$

Для каждой вершины v и каждого истока i выполняется:

$$\forall v: \quad \sum_{e \in In(v)} f_e^i - \sum_{e \in Out(v)} f_e^i = \begin{cases} d_{iv}, & v \neq i, \\ -\sum_j d_{ij}, & v = i. \end{cases} \tag{4}$$

где f_e^i — поток по дуге e, направленный к стоку из истока i.

В промежуточных узлах сколько машин въехало, столько же и выехало.

Матрицы

Определение 2 (Матрица инцидентности)**. Матрица инцидентности** A ориентированного графа $G = (V, E)$ определяется следующим образом:

- Размерность: $A \in \mathbb{R}^{n \times m}$, где
  - $n = |V|$ — число вершин
  - $m = |E|$ — число рёбер
- Строки соответствуют вершинам $i = 1, \ldots, n$
- Столбцы соответствуют рёбрам $e = 1, \ldots, m$
- Для каждого ориентированного ребра $e = (p \to q)$:

$$
\begin{aligned}
&A_{pe} = -1, \\
&A_{qe} = +1, \\
&A_{ie} = 0 \quad \text{для всех } i \neq p, q.
\end{aligned}
$$

	e_1	e_2	e_3	e_4	e_5
v_1	-1	0	$+1$	0	0
v_2	$+1$	-1	0	-1	0
v_3	0	$+1$	-1	0	$+1$
v_4	0	0	0	$+1$	-1

Определение 3 (Матрица потоков)**.** Матрица потоков F ориентированного графа $G = (V, E)$ определяется следующим образом:

- Размерность: $F \in \mathbb{R}^{m \times n}$, где
  - $m = |E|$ — число рёбер графа,
  - $n = |V|$ — число вершин графа.

$$
F = \left[f^1 \mid f^2 \mid \cdots \mid f^n \right], \quad f^j = \sum_{i=1}^{n} f^{(i,j)}
$$

	Назначение 1	Назначение 2	$\cdots$	Назначение n
Дуга 1	f_1^1	f_1^2	$\cdots$	f_1^n
Дуга 2	f_2^1	f_2^2	$\cdots$	f_2^n
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
Дуга m	f_m^1	f_m^2	$\cdots$	f_m^n

Свойства:

1. Каждый столбец f^j соответствует всем потокам, направленным в узел j
2. Сумма элементов в строке e даёт полный поток по дуге e: $\sum_j F_{ej} = x_e$
3. Сумма элементов в столбце j: $\sum_e F_{ej} = \sum_i d_{ij}$ (общий спрос на узел j)

Функции и переменные: Функция затрат (BPR-функция)

$$t_e(x_e) = t_e^0 \left(1 + \alpha \left(\frac{x_e}{C_e}\right)^\beta\right) \tag{5}$$

где:

- t_e^0 — время проезда по ребру e при свободном потоке
- C_e — пропускная способность ребра e
- α, β — калибровочные параметры (типичные значения: $\alpha = 0.15$, $\beta = 4$)

Интуиция: чем больше поток x_e, тем выше стоимость проезда $t_e(x_e)$.

1.4 Седловой подход (Primal–Dual)

Классическая постановка задачи Задача TAP в постановке User Equilibrium может быть представлена как задача минимизации (формула Бекмана):

$$\min_{x \geq 0} \sum_{e \in E} \int_0^{x_e} t_e(w)\, dw \tag{6}$$

Позднее будут приведены требуемые ограничения для окончательной постановки задачи

Седловая формулировка Седловой (primal–dual) подход позволяет переформулировать задачу как:

$$\underbrace{L(x, \mu)}_{\text{Лагранжиан}} = \underbrace{\sum_{e \in E} \int_0^{x_e} t_e(u)\, du}_{\text{Целевая функция (сумма интегралов задержек)}} + \underbrace{\mu^\top (Ax + b)}_{\text{Ограничения баланса потоков}} \tag{7}$$

где μ — вектор двойственных переменных (потенциалов узлов, dual).

2 Основная часть

2.1 Формальная постановка Traffic Assignment Problem

Traffic Assignment Problem (TAP) – это задача распределения заданного спроса (по OD-парам) по транспортной сети. В состоянии **пользовательского равновесия (User Equilibrium, UE)** все использующиеся маршруты каждой OD-пары имеют равные и минимальные затраты (время) по сравнению с альтернативными маршрутами. Wardrop сформулировал это как принцип: никакой отдельный водитель не может уменьшить время пути, переключившись на другой маршрут.

Бекман, МакГуир и Уинстен (1956) показали, что условия UE можно сформулировать как решение задачи минимизации. **Функция Бекмана** – это целевая функция, равная сумме интегралов от нуля до потока по функции затрат на каждой дуге сети. Для каждого ребра a с потоком x_a и зависимостью времени $t_a()$ функция Бекмана имеет вид:

$$Z(x) = \sum_{a \in A} \int_0^{x_a} t_a(u) du,$$

где сумма берётся по всем дугам A сети. **Статическое пользовательское равновесие** при фиксированном спросе эквивалентно решению задачи минимизации $Z(x)$ при ограничениях сохранения потока: суммарный отправленный поток по каждой OD-паре равен её спросу, а поток на дуге равен сумме потоков маршрутов, проходящих через эту дугу.

Таким образом, решение задачи минимизирует суммарные "потенциалы" на дугах, и точка минимума удовлетворяет критерию равенства затрат на всех задействованных маршрутах (условия равновесия Уордропа). Бекман показал эквивалентность этой формулировки и условий UE, а также то, что такое решение единственно по потокам на дугах. В практических приложениях для решения этой выпуклой оптимизационной задачи часто применялся алгоритм Франка–Вульфа (FW) либо метод последовательных усреднений (MSA).

2.2 Предпосылки существования и единственности решения

Корректность решения UE зависит от свойств функций затрат на дугах. **Непрерывность функций затрат** (т.е. плавная зависимость времени t_a от потока x) является достаточным условием существования по крайней мере одного решения равновесия Уордрола. Интуитивно, если затраты изменяются непрерывно, то всегда можно "настроить" распределение потоков так, чтобы спрос удовлетворялся и не осталось стимулов к одностороннему изменению маршрута.

Монотонность (неубывание) функций затрат обеспечивает единственность решения (в терминах потоков на дугах). Строго монотонные неубывающие функции (например, типичные BPR-функции для дорожных нагрузок) делают функцию Бекмана строго выпуклой, а значит задача имеет единственное оптимальное решение. Это га-

рантирует, что равновесные потоки на каждом ребре определены однозначно (хотя сами маршруты и разбиение потока по равнозначным маршрутам могут быть неединственными). Таким образом, **непрерывность гарантирует существование, а строгая монотонность – единственность равновесия**. В случае нарушения этих предпосылок (например, немонотонные зависимости затрат) возможно возникновение нескольких равновесных состояний или отсутствие стабильного равновесия.

2.3 Седловая формулировка через лагранжиан ($\min_x \max_\lambda L(x, \lambda)$)

Задачу поиска UE можно записать в седловой форме как задачу минимизации–максимизации лагранжиана. Примем переменные x – как первичные (праймал) переменные, представляющие потоки на дугах (или маршрутах), а переменные λ– как двойственные переменные, связанные с ограничениями (например, баланса потока для каждой O-D пары или узла). Для любой вершины $i \in V$ выполняется баланс:

$$\sum_{e \in In(i)} f_e^j - \sum_{e \in Out(i)} f_e^j = \begin{cases} -d_{ij}, & \text{если } i \neq j, \\ \sum_k d_{kj}, & \text{если } i = j. \end{cases} \tag{1}$$

Рассматривая левую часть уравнения (1), можно заметить, что она представляет собой действие матрицы инцидентности A на вектор f^j:

$$(Af^j)_i = \sum_{e \in In(i)} f_e^j - \sum_{e \in Out(i)} f_e^j.$$

Тогда исходное уравнение (1) можно переписать в компактной матричной форме:

$$(Af^j)_i = \begin{cases} -d_{ij}, & i \neq j, \\ \sum_k d_{kj}, & i = j. \end{cases}$$

2

2.3.1 Вектор λ^j и матрица Λ_d

Представим уравнение (2) в виде векторного уравнения:

$$Af^j + \lambda^j = 0, \tag{3}$$

где $\lambda^j \in \mathbb{R}^n$ — вспомогательный вектор. Из (3) по координатам:

$$(Af^j)_i + \lambda_i^j = 0 \quad \Rightarrow \quad \lambda_i^j = -(Af^j)_i.$$

Используя (2), получаем явный вид λ^j:

$$\lambda_i^j = \begin{cases} d_{ij}, & i \neq j \,(\text{минус } -d_{ij}), \\ -\sum_k d_{kj}, & i = j \,(\text{минус } \sum_k d_{kj}). \end{cases} \tag{4}$$

Теперь построим матрицу $\Lambda_d \in \mathbb{R}^{n\times n}$, в которой j-й столбец — это λ^j:

$$\Lambda_d := \begin{bmatrix} \lambda^1 & \lambda^2 & \cdots & \lambda^n \end{bmatrix}.$$

По координатам это означает:

$$(\Lambda_d)_{ij} = \lambda_i^j.$$

Подставляя (4), получаем для Λ_d:

$$(\Lambda_d)_{ij} = \begin{cases} d_{ij}, & i \neq j, \\ -\sum_k d_{kj}, & i = j. \end{cases} \tag{5}$$

То есть:

- Все элементы вне диагонали: $(\Lambda_d)_{ij} = d_{ij}$;
- Диагональные элементы: $(\Lambda_d)_{jj} = -\sum_k d_{kj}$.

2.3.2 Матричное уравнение баланса

Теперь соберём все уравнения (3) по всем j. Напомним, что

$$F = [f^1 \dots f^n].$$

Матрица AF имеет элементы

$$(AF)_{ij} = (Af^j)_i.$$

Поэтому уравнения $Af^j + \lambda^j = 0$ для всех j эквивалентны одной матричной записи:

$$AF + \Lambda_d = 0. \tag{6}$$

Рассмотрим как будет выглядеть матрица Λ_d.

2.3.3 Связь с матрицей спроса D

Сначала определимся с матрицей спроса D:

- Размер $n \times n$;
- Строка i соответствует узлу-источнику (origin);
- Столбец j соответствует узлу-назначению (destination);
- Элемент d_{ij} — объём транспорта, который необходимо перевезти из узла i в узел j.

Обычно предполагают, что нет поездок “из узла в него же”, поэтому

$$d_{ii} = 0.$$

Теперь покажем, что матрица Λ_d совпадает с формулой

$$\Lambda_d = D - \mathrm{diag}(D^\top 1),$$

где 1 — вектор из единиц длины n.

Вектор столбцовых сумм спроса Рассмотрим вектор

$$s := D^\top 1 \in \mathbb{R}^n.$$

Его j-я компонента:

$$s_j = (D^\top 1)_j = \sum_i D_{ij} = \sum_i d_{ij}$$

— это сумма по столбцу j матрицы D (все потоки, которые приходят в j из разных источников).

Диагональная матрица из этих сумм Строим диагональную матрицу:

$$\mathrm{diag}(s) = \mathrm{diag}(D^\top 1) = \begin{pmatrix} s_1 & 0 & \cdots & 0 \\ 0 & s_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & s_n \end{pmatrix}.$$

Её элементы:

$$(\mathrm{diag}(D^\top 1))_{ij} = \begin{cases} s_j = \sum_k d_{kj}, & i = j, \\ 0, & i \neq j. \end{cases}$$

Разность $D - \mathbf{diag}(D^\top 1)$ Теперь посмотрим на матрицу

$$\tilde{\Lambda} := D - \mathrm{diag}(D^\top 1).$$

Её элементы:

- Если $i \neq j$:
$$\tilde{\Lambda}_{ij} = D_{ij} - 0 = d_{ij},$$
- Если $i = j$:
$$\tilde{\Lambda}_{jj} = D_{jj} - (D^\top 1)_j = d_{jj} - \sum_k d_{kj} = -\sum_k d_{kj}.$$

что в точности совпадает с формулой для диагональных элементов Λ_d.

Результат Таким образом, матрица $\tilde{\Lambda}$, построенная по формуле $D - \mathrm{diag}(D^\top 1)$, имеет те же элементы, что и Λ_d, введённая через балансные уравнения. Следовательно:

$$\Lambda_d = D - \mathrm{diag}(D^\top 1),$$

а уравнение баланса можно компактно записать как:

$$AF + \Lambda_d = 0.$$

Это важно для диагональных элементов матрицы Λ_d.

$$\Lambda_d = D - \mathrm{diag}(D^\top 1),$$

а его элементы задаются формулой (5).

2.3.4 Агрегация по назначениям

В практической постановке Бекмана нас интересует суммарный поток по каждой дуге:

$$x_e = \sum_j f_e^j.$$

В матричном виде:

$$x = F1,$$

где $1 \in \mathbb{R}^n$ — столбец из единиц.
Теперь применим (6) и домножим это равенство справа на 1::

$$AF1 + \Lambda_d 1 = 0. \tag{7}$$

По определениям:

- $F1 = x$;
- $\Lambda_d 1 =: b$ — определение вектора b.

То есть (7) переписывается как

$$Ax + b = 0. \tag{8}$$

Рассмотрим $b = \Lambda_d 1$. Компонента b_i:

$$b_i = (\Lambda_d 1)_i = \sum_j (\Lambda_d)_{ij}.$$

Используем (5):

$$b_i = \sum_j (\Lambda_d)_{ij} = \sum_{j \neq i} d_{ij} + \left(- \sum_k d_{ki} \right) = \sum_j d_{ij} - \sum_k d_{ki}. \tag{9}$$

Интерпретация:

- $\sum_j d_{ij}$ — сколько узел i хочет отправить (суммарный исходящий спрос);
- $\sum_k d_{ki}$ — сколько в узел i хотят отправить (суммарный входящий спрос).

Значит

$$b_i = (\text{исходящий спрос из } i) - (\text{входящий спрос в } i).$$

Если $b_i > 0$, узел i — чистый источник; если $b_i < 0$, узел i — чистый сток.
Из балансных уравнений по назначениям, умножая на вектор единиц, получаем:

$$x = F1, \quad b = \Lambda_d 1, \quad Ax + b = 0,$$

где

$$b_i = \sum_i d_{ij} - \sum_h d_{ki}.$$

2.3.5 Построение лагранжиана

Для задачи вида

$$\min_x f(x) \quad \text{при } g(x) = 0,$$

лагранжиан имеет вид

$$L(x, \mu) = f(x) + \mu^\top g(x),$$

где μ — вектор множителей Лагранжа.
У нас:

- $f(x) = \Phi(x)$;
- $g(x) = Ax + b$.

Значит, по определению,

$$L(x, \mu) = \Phi(x) + \mu^\top (Ax + b).$$

Это и будет лагранжиан.

2.3.6 Явный вид лагранжиана в векторной форме

Подставим $\Phi(x)$:

$$L(x,\mu) = \sum_{e\in E} \int_0^{x_e} t_e(u)du + \mu^\top (Ax + b). \tag{12}$$

Теперь распишем $\mu^\top(Ax+b)$ подробнее. Разобьём:

$$\mu^\top(Ax+b) = \mu^\top Ax + \mu^\top b.$$

Скаляр $\mu^\top Ax$ можно переписать как $(A^\top\mu)^\top x$:

$$\mu^\top Ax = (A^\top\mu)^\top x = \sum_{e\in E} (A^\top\mu)_e x_e.$$

Введём обозначение:

$$(A^\top\mu)_e =: \alpha_e.$$

Тогда

$$\mu^\top Ax = \sum_e \alpha_e x_e. \tag{14}$$

Дальше распишем α_e через μ и структуру A.
Напомним: для дуги $e = (i \to j)$

$$A_{ie} = -1, \quad A_{je} = +1, \quad \text{остальные } A_{ke} = 0.$$

Тогда

$$\alpha_e = (A^\top\mu)_e = \sum_k A_{ke}\mu_k = (-1)\mu_i + (+1)\mu_j = \mu_j - \mu_i. \tag{15}$$

Подставляем (15) в (14):

$$\mu^\top Ax = \sum_{e=(i\to j)\in E} (\mu_j - \mu_i)x_{ij}. \tag{16}$$

В результате лагранжиан (12) принимает вид:

$$L(x,\mu) = \sum_{(i,j)\in E} \int_0^{x_{ij}} t_{ij}(u)du + \sum_{(i,j)\in E} (\mu_j - \mu_i)x_{ij} + \sum_{i\in V} \mu_i b_i. \tag{17}$$

Это полностью развернутая по дугам и вершинам форма.
Можно сгруппировать члены по дугам:

$$L(x,\mu) = \sum_{(i,j)\in E} \left[\int_0^{x_{ij}} t_{ij}(u)du + (\mu_j - \mu_i)x_{ij} \right] + \sum_{i\in V} \mu_i b_i. \tag{18}$$

2.3.7 Интерпретация двойственных переменных.

Условия оптимальности седловой точки дают интерпретацию двойственных переменных λ (или узловых потенциалов μ), связанных со стоимостью пути. Уравнения стационарности $\partial L/\partial x = 0$ приводят к условиям равновесия: для каждой дуги (i,j) с ненулевым потоком $x_{ij} > 0$ должно выполняться

$$t_{ij}(x_{ij}) = \mu_i - \mu_j,$$

то есть разность потенциалов узлов равна предельным затратам на дуге. В противном случае (если дуга не используется, $x_{ij} = 0$) выполняется неравенство

$$t_{ij}(x_{ij}) \geq \mu_i - \mu_j,$$

которое предотвращает улучшение при запуске положительного потока по этой дуге.

Эти условия эквивалентны условиям оптимальности Беллмана: потенциал μ_i можно интерпретировать как (обобщённую) стоимость достижения некоторого эталонного узла, и выполняются неравенства

$$\mu_i \leq \mu_j + t_{ij}(x_{ij}) \quad \text{для всех рёбер } (i,j),$$

причём на рёбрах, по которым протекает положительный поток, достигается равенство. Если рассматривать μ_r и μ_s как потенциалы соответствующих исходного и конечного пунктов, то разность

$$\mu_r - \mu_s$$

можно трактовать как *стоимость отправки единицы потока из r в s*, то есть равновесное время пути между r и s. Таким образом, двойственные переменные λ (или узловые потенциалы μ) в седловой постановке имеют ясный физический смысл: в оптимуме они совпадают с кратчайшими путевыми затратами между соответствующими узлами или O–D-парами.

В рамках седловой задачи ищется пара (x^*, λ^*) (седловая точка), удовлетворяющая

$$\min_x \max_\lambda L(x,\lambda),$$

так что x^* минимизирует лагранжиан при фиксированных потенциалах λ^*, а λ^* максимизируют его при фиксированном x^*, обеспечивая выполнение ограничений и интерпретацию λ^* как равновесных путевых затрат.

2.4 Праймал–дуальная итеративная схема решения

Седловую задачу можно решать итеративно, чередуя обновление *праймальных переменных* (потоков x) и *дуальных переменных* (потенциалов λ или μ). Идея состоит в том, чтобы постепенно приближаться к условиям ККТ (условия ККТ — набор необходимых, а при выпуклости и достаточных, условий оптимальности для задачи с ограничениями, включающих стационарность, допустимость и дополняющую нежёсткость): на праймальном шаге мы стремимся уменьшить целевую функцию (распределяем потоки по кратчайшим путям при текущих временах/потенционах), а на дуальном шаге — корректируем потенциалы так, чтобы лучше удовлетворять ограничения сохранения потока.

2.4.1 Алгоритм

Инициализация. Задаём начальное распределение потоков, например

$$x_a^{(0)} = 0 \quad \text{для всех дуг } a,$$

задаём начальные потенциалы (например, $\mu^{(0)} = 0$) и номер итерации $k = 0$.

Повторять до сходимости:

1. **Обновление затрат.** Вычислить текущие времена/затраты на дугах

$$t_a^{(k)} = t_a\big(x_a^{(k)}\big), \qquad \forall a.$$

2. **Праймальный шаг (распределение по кратчайшим путям).** Для каждой O–D пары (r, s):

   (a) найти кратчайший путь $P_{rs}^{(k)}$ в сети с весами $t_a^{(k)}$;

   (b) обозначить $d_{rs}^{(k)} = t\big(P_{rs}^{(k)}\big)$ как длину (общее время) этого пути;

   (c) если используется схема типа *all-or-nothing*, назначить весь спрос Q_{rs} по маршруту $P_{rs}^{(k)}$.

   В результате получаем вспомогательное распределение потоков $y^{(k)}$ по дугам (по принципу “всё по кратчайшим”).

3. **Корректировка потоков.** Обновить x путём смешивания старых и новых потоков:

$$x^{(k+1)} := x^{(k)} + \alpha_k\big(y^{(k)} - x^{(k)}\big),$$

   где шаг $\alpha_k \in (0, 1]$ может задаваться, например,

   - по правилу метода последовательных усреднений (MSA):

$$\alpha_k = \frac{1}{k+1},$$

- или как результат линейного поиска вдоль направления $y^{(k)} - x^{(k)}$, минимизирующего функционал Бекмана $Z(x)$ (вариант алгоритма Франка–Вульфа).

4. **Дуальный шаг (обновление потенциалов).** При наличии явной дуальной схемы потенциалы можно обновлять, исходя из остатка по ограничениям сохранения потока. Пусть A — матрица инцидентности, а b — вектор нетто-спроса узлов (источники/стоки), так что баланс записывается как

$$Ax + b = 0.$$

Тогда можно использовать шаг вида

$$\mu_i^{(k+1)} := \mu_i^{(k)} + \rho_k\big((Ax^{(k+1)})_i + b_i\big),$$

где $(Ax^{(k+1)})_i = \sum_{a \in In(i)} x_a^{(k+1)} - \sum_{a \in Out(i)} x_a^{(k+1)}$ и $\rho_k > 0$ — шаг по дуальным переменным.

Во многих классических схемах (Франк–Вульф, MSA) распределение по O–D парам сразу удовлетворяет спросу (all-or-nothing даёт полный спрос для каждой пары), поэтому ограничения выполняются на уровне O–D и потенциалы можно *неявно* восстанавливать через длины кратчайших путей: на равновесии

$$d_{rs} \approx \mu_r - \mu_s.$$

5. Увеличить номер итерации: $k := k + 1$ и перейти к шагу 1.

Комментарий. Шаг 2 фактически решает вспомогательную задачу поиска кратчайших путей (минимизирует лагранжиан по x при фиксированных потенциалах/затратах). При использовании алгоритма Франка–Вульфа шаг 3 реализуется через линейный поиск по α_k, обеспечивая монотонное уменьшение функционала Бекмана $Z(x)$. В методе последовательных усреднений (MSA) используется убывающая последовательность шагов $\alpha_k = 1/(k+1)$ без явного поиска.

Дуальный шаг 4 в явном виде особенно важен для алгоритмов, основанных на проекциях или градиентных методах в пространстве потенциалов, когда ограничения сохранения потока не удовлетворяются точно на каждом праймальном шаге. В стандартных же FW/MSA-схемах спрос удовлетворяется на уровне O–D матрицы, и узловые потенциалы могут быть восстановлены из равновесных длин кратчайших путей d_{rs}.

По сути, итеративная схема чередует *решение задачи на кратчайших путях и корректировку распределения потоков*. С каждой итерацией потоки x приближаются к оптимальному распределению: неиспользуемые маршруты оказываются (в терминах времени) не короче используемых, а времена на маршрутах, по которым идёт поток, выравниваются. Алгоритм завершается, когда выполняются критерии сходимости (например, когда

разность между стоимостью лучшего альтернативного пути и текущего используемого пути по каждой O–D паре достаточно мала).

2.5 Критерии остановки алгоритма

Чтобы определить, достигнуто ли равновесие, применяются различные критерии остановки. В практике решения задач пользовательского равновесия (UE) наиболее распространены следующие показатели.

2.5.1 UE-gap (разрыв равновесия)

UE-gap измеряет отклонение текущего решения от условий равновесия. Часто используется относительный разрыв вида

$$\mathrm{RGAP}(x) = \frac{\sum\limits_{(r,s)} Q_{rs}\big(T_{rs}(x) - T^{\star}_{rs}(x)\big)}{\sum\limits_{(r,s)} Q_{rs}\, T^{\star}_{rs}(x)},$$

где

- Q_{rs} — спрос между O–D парой (r, s),
- $T_{rs}(x)$ — среднее время пути (или стоимость) при текущем распределении потоков x для пары (r, s),
- $T^{\star}_{rs}(x)$ — минимально возможное время (стоимость) для пары (r, s) в текущей сетевой конфигурации, то есть длина кратчайшего пути при текущих дуговых затратах.

В равновесии для всех используемых маршрутов выполняется $T_{rs}(x) = T^{\star}_{rs}(x)$, поэтому $\mathrm{RGAP}(x) = 0$. Если разрыв становится меньше заданного порога (например, 10^{-6}), текущее решение обычно считается достаточно точным.

2.5.2 Седловой разрыв (duality gap)

Седловой (или двойственный) разрыв определяется как разность между значением праймальной целевой функции (функционала Бекмана, то есть суммарных интегралов затрат по дугам) и значением двойственной функции (суммарной “стоимости” по потенциалам). В оптимуме эти значения совпадают, а разность всегда неотрицательна. Уменьшение duality gap и достижение малой величины используется как строгий критерий близости к оптимуму.

2.5.3 Нарушение ограничений

Проверяется выполнение ограничений баланса и других ограничений модели, например:

- суммарный поток, исходящий из каждого источника, равен соответствующему спросу;
- суммарный поток, входящий в каждый сток, соответствует спросу;
- потоки неотрицательны и удовлетворяют, при необходимости, ограничениям по пропускной способности.

В равновесии все эти ограничения должны выполняться (с учётом численных погрешностей). Если обнаруживаются заметные нарушения (например, какой-то спрос существенно не удовлетворён), алгоритм считается несошедшимся.

2.5.4 Относительные изменения между итерациями

Сравниваются решения на текущей и предыдущей итерациях. Типичный критерий:

$$\max_a \left| x_a^{(k+1)} - x_a^{(k)} \right| \le \varepsilon_x,$$

где ε_x — заданный порог. Аналогично можно отслеживать изменения потенциалов или целевой функции $Z(x)$ (функционала Бекмана), либо разницы в UE-gap между итерациями. Если изменения становятся пренебрежимо малыми, решение считается стабилизировавшимся.

2.5.5 Ограничение по числу итераций

Задаётся максимальное число итераций $K_{\max}$, после которого алгоритм останавливается независимо от текущих значений критериев. Это гарантирует завершение вычислений. Если к моменту достижения $K_{\max}$ критерии сходимости ещё не выполнены, текущее решение принимается как приближённое.

На практике обычно комбинируют несколько критериев, например: требование к малому UE-gap *или* достижение лимита по числу итераций. Это предотвращает чрезмерно долгую работу алгоритма при асимптотически затухающем улучшении.

2.6 Алгоритмы Беллмана–Форда и Дейкстры для кратчайших путей

В шаге поиска кратчайших путей (см. праймал–дуальный алгоритм) могут использоваться различные классические алгоритмы. Наиболее известны алгоритм Беллмана–Форда и алгоритм Дейкстры. Выбор зависит от свойств дуговых затрат (например, наличия/отсутствия отрицательных весов).

2.6.1 Алгоритм Беллмана–Форда

Алгоритм Беллмана–Форда (BF) находит кратчайшие пути от заданного источника s до всех вершин даже при наличии рёбер с отрицательными весами (при отсутствии

отрицательных циклов). Для дорожных сетей времена поездки обычно неотрицательны, но BF полезен своей простотой и тем, что легко параллелизуется.

Идея алгоритма следующая: выполняются многократные обходы всех рёбер графа с операцией *релаксации*. После k-й итерации внешнего цикла гарантируется, что найдены наилучшие пути, использующие не более k рёбер. Так как любой простой путь не может содержать более $|V| - 1$ рёбер, достаточно выполнить $|V| - 1$ проходов по рёбрам, чтобы получить точные кратчайшие расстояния при отсутствии отрицательных циклов.

Базовая операция — релаксация ребра (v, u):

$$\text{если } \mathrm{dist}[u] > \mathrm{dist}[v] + w(v, u), \text{ то улучшить } \mathrm{dist}[u].$$

Простейшая реализация алгоритма Беллмана–Форда:

```
def Relax(v, u, w, dist, prev):
    if dist[u] > dist[v] + w(v, u):
        dist[u] = dist[v] + w(v, u)
        prev[u] = v
        return True
    return False

def BellmanFord(G, s):
    dist = {v: inf for v in G.V}
    prev = {v: None for v in G.V}
    dist[s] = 0
    for i in range(0, len(G.V) - 1):  # |V| - 1 итераций
        for (v, u) in G.E:
            Relax(v, u, G.w, dist, prev)
    return dist, prev
```

Здесь $\mathrm{dist}[v]$ — текущая оценка кратчайшего расстояния от s до вершины v, а $\mathrm{prev}[v]$ хранит предыдущую вершину на пути.

Выявление отрицательных циклов. Дополнительный проход по всем рёбрам позволяет обнаружить наличие отрицательных циклов. Если после $|V| - 1$ итераций есть ребро, которое может быть ещё релаксировано, это означает существование цикла отрицательного веса.

```
def HasNegativeCycle(G, s):
    dist = {v: inf for v in G.V}
    prev = {v: None for v in G.V}
    dist[s] = 0
    # Основной цикл Беллмана-Форда
```

```
for i in range(0, len(G.V) - 1):
    for (v, u) in G.E:
        Relax(v, u, G.w, dist, prev)
# Дополнительный проход для проверки
for (v, u) in G.E:
    if Relax(v, u, G.w, dist, prev):
        return True
return False
```

Асимптотическая сложность алгоритма Беллмана–Форда составляет

$$O(|V| \cdot |E|).$$

2.6.2 Теорема Беллмана о корректности алгоритма

Теорема 1 (Беллман, 1958)**.** *Если в графе G отсутствуют циклы отрицательного веса, то по завершении работы алгоритма Беллмана–Форда после $|V| - 1$ итераций выполняется*

$$\forall v \in V : \quad \text{dist}[v] = \rho(s, v),$$

где $\rho(s, v)$ — длина кратчайшего пути от s до v.

Доказательство. Рассмотрим произвольную вершину v и некоторый кратчайший путь от s до v:

$$P(s, v) = (v_0 = s, v_1, \ldots, v_{n-1}, v_n = v).$$

Докажем по индукции по k, что после k-й итерации внешнего цикла выполняется

$$\text{dist}[v_k] = \rho(s, v_k).$$

База. Для $k = 0$ имеем $v_0 = s$, инициализация даёт $\text{dist}[s] = 0 = \rho(s, s)$.

Переход. Предположим, что после $(k-1)$-й итерации $\text{dist}[v_{k-1}] = \rho(s, v_{k-1})$. На k-й итерации в ходе полного обхода рёбер будет рассмотрено ребро (v_{k-1}, v_k). Поскольку

$$\rho(s, v_k) = \rho(s, v_{k-1}) + w(v_{k-1}, v_k),$$

операция релаксации для этого ребра установит

$$\text{dist}[v_k] \le \text{dist}[v_{k-1}] + w(v_{k-1}, v_k) = \rho(s, v_{k-1}) + w(v_{k-1}, v_k) = \rho(s, v_k).$$

С другой стороны, алгоритм никогда не занижает расстояния: $\text{dist}[v]$ всегда является верхней оценкой длины некоторого пути от s до v, следовательно, $\text{dist}[v_k] \ge \rho(s, v_k)$. Отсюда $\text{dist}[v_k] = \rho(s, v_k)$.

Так как любой кратчайший путь имеет длину не более $|V| - 1$ рёбер, после $|V| - 1$ итераций корректные расстояния получены для всех вершин. □

2.6.3 Теорема о детектировании отрицательных циклов

Теорема 2 (Беллман, 1958)**.** *Алгоритм с дополнительным проходом по рёбрам корректно определяет наличие или отсутствие цикла отрицательного веса: если после* $|V| - 1$ *итераций существует ребро, которое можно ещё релаксировать, то в графе существует цикл отрицательного веса, и наоборот.*

Доказательство. Первая часть следует из предыдущей теоремы: если отрицательных циклов нет, то после $|V| - 1$ итераций значения $\mathrm{dist}[v]$ уже равны длинам кратчайших путей, поэтому дальнейшие релаксации невозможны.

Предположим теперь, что имеется цикл отрицательного веса

$$C = (v_0, v_1, \ldots, v_{n-1}, v_n = v_0),$$

и алгоритм не сигнализирует о нём (то есть ни одно ребро этого цикла не приводит к улучшению расстояния на дополнительном проходе). Тогда для всех k выполнено неравенство

$$\mathrm{dist}[v_{k+1}] \le \mathrm{dist}[v_k] + w(v_k, v_{k+1}).$$

Суммируя по всем рёбрам цикла,

$$\sum_{k=0}^{n-1} \mathrm{dist}[v_{k+1}] \le \sum_{k=0}^{n-1} \mathrm{dist}[v_k] + \sum_{k=0}^{n-1} w(v_k, v_{k+1}).$$

Так как $v_n = v_0$, левая и правая суммы по расстояниям совпадают и сокращаются, остаётся

$$0 \le \sum_{k=0}^{n-1} w(v_k, v_{k+1}),$$

что противоречит предположению о том, что цикл имеет отрицательный вес. Следовательно, наличие отрицательного цикла обязательно приводит к возможности дальнейшей релаксации хотя бы одного ребра. □

2.6.4 Алгоритм Дейкстры

Для дорожных сетей веса рёбер (время, затраты) обычно неотрицательны, и в этом случае чаще используется алгоритм Дейкстры, обладающий более выгодной асимптотической сложностью. Алгоритм Дейкстры реализует жадную стратегию: на каждом шаге выбирается вершина с минимальным текущим расстоянием, ещё не помеченная как окончательно обработанная, после чего выполняется релаксация всех исходящих из неё рёбер.

Одна из типичных реализаций алгоритма Дейкстры (с приоритетной очередью):

```
import heapq

def Dijkstra(G, s):
```

```
dist = {v: inf for v in G.V}
prev = {v: None for v in G.V}
dist[s] = 0
heap = [(0, s)]  # (расстояние, вершина)
in_heap = {v: True for v in G.V}

while heap:
    d_v, v = heapq.heappop(heap)
    if not in_heap[v]:
        continue
    in_heap[v] = False
    for u in G.neighbors(v):
        if in_heap[u] and dist[u] > dist[v] + G.w(v, u):
            dist[u] = dist[v] + G.w(v, u)
            prev[u] = v
            heapq.heappush(heap, (dist[u], u))
return dist, prev
```

При использовании двоичной кучи (бинарной пирамиды) сложность алгоритма составляет

$$O(|E| \log |V|).$$

При использовании более сложных структур данных (например, фибоначчиевой кучи) можно получить оценку

$$O(|E| + |V| \log |V|).$$

Таким образом, для неотрицательных дуговых затрат алгоритм Дейкстры обычно предпочтительнее с точки зрения вычислительной эффективности, тогда как алгоритм Беллмана–Форда обладает преимуществом простоты и хорошо подходит для распараллеливания (каждый проход по рёбрам может выполняться параллельно).

2.6.5 Теорема Дейкстры (1959)

Теорема 3 (Дейкстра, 1959)**.** *Пусть все рёбра ориентированного графа $G = (V, E)$ имеют неотрицательные веса $w(e) \geq 0$. Тогда по завершении работы алгоритма Дейкстры, запущенного из источника s, для всех вершин $v \in V$ выполняется*

$$d[v] = \rho(s, v),$$

где $\rho(s, v)$ — длина кратчайшего пути из s в v, а $d[v]$ — метка расстояния, вычисленная алгоритмом.

Доказательство (индукция по числу итераций). Обозначим через S множество вершин,

для которых алгоритм уже зафиксировал окончательные значения $d[\cdot]$ (эти вершины извлечены из очереди), а через $U = V \setminus S$ — оставшиеся вершины.

Инвариант алгоритма: *для всех $v \in S$ метка $d[v]$ равна $\rho(s, v)$, и для всех $u \in V$ выполняется $d[u] \geq \rho(s, u)$ (метки — верхние оценки кратчайших расстояний).*

База. В начале работы $S = \emptyset$, $d[s] = 0$, $d[v] = +\infty$ для $v \neq s$. После первой итерации в S попадает вершина s с $d[s] = 0$, и очевидно

$$d[s] = 0 = \rho(s, s).$$

Переход. Предположим, что после некоторой итерации инвариант выполнен: для всех $v \in S$ имеем $d[v] = \rho(s, v)$. На следующем шаге алгоритм выбирает вершину $u \in U$ с минимальной меткой $d[u]$ и переносит её в S.

Рассмотрим кратчайший путь $P(s, u)$ из s в u. Пусть этот путь впервые покидает множество S в вершине y, а предшествующая ей вершина x лежит в S:

$$P(s, u) = (s = v_0, \ldots, v_k = x, v_{k+1} = y, \ldots, u), \quad x \in S,\ y \in U.$$

Тогда

$$\rho(s, y) = \rho(s, x) + w(x, y).$$

По предположению индукции $\rho(s, x) = d[x]$, а так как $w(x, y) \geq 0$, то при добавлении x в множество S алгоритм выполнил релаксацию ребра (x, y) и установил

$$d[y] \leq d[x] + w(x, y) = \rho(s, x) + w(x, y) = \rho(s, y).$$

С другой стороны, по инварианту метки не занижают истинные расстояния: $d[y] \geq \rho(s, y)$. Следовательно,

$$d[y] = \rho(s, y).$$

Алгоритм выбирает u как вершину с минимальной меткой в U, поэтому

$$d[u] \leq d[y] = \rho(s, y).$$

С другой стороны, любой путь из s в u должен содержать хотя бы одну вершину из U (в частности, y), а так как все веса неотрицательны, то

$$\rho(s, u) \geq \rho(s, y).$$

Используя факт, что $d[u]$ всегда является верхней оценкой для $\rho(s, u)$, получаем цепочку:

$$\rho(s, u) \leq d[u] \leq d[y] = \rho(s, y) \leq \rho(s, u).$$

Отсюда следует равенство $d[u] = \rho(s, u)$, что завершает шаг индукции.

Таким образом, после завершения алгоритма для всех вершин v выполняется $d[v] =$

$\rho(s, v)$. □

2.6.6 Восстановление кратчайших маршрутов

Восстановление маршрутов после выполнения алгоритмов Беллмана–Форда или Дейкстры осуществляется с помощью хранения *предшественников*. Каждый раз при успешной релаксации ребра (i, j) (когда обновляется метка расстояния для вершины j) сохраняется, что оптимальный предшественник j равен i:

$$\text{если } d[j] > d[i] + w(i, j), \text{ то установить } \mathrm{pred}[j] := i.$$

После завершения алгоритма кратчайший путь из источника s до вершины t восстанавливается путём обратного прохода по цепочке предшественников:

$$t,\ \mathrm{pred}[t],\ \mathrm{pred}[\mathrm{pred}[t]],\ \ldots,\ s.$$

Развернув эту последовательность, получаем явный маршрут с минимальной суммарной длиной.

2.6.7 Batch-обработка множества O–D пар

В задачах распределения потоков часто требуется найти кратчайшие пути для множества O–D пар (в идеале — для всех пар вершин). Возможны следующие подходы.

- **Многократный запуск алгоритма Дейкстры.** Если исходов R относительно немного, алгоритм Дейкстры запускается для каждой исходной вершины r отдельно. Тогда суммарная сложность составляет порядка

  $$R \cdot O(|E| \log |V|).$$

  Этот подход хорошо работает для небольшого числа источников.

- **Алгоритмы all-pairs.** Для нахождения кратчайших путей между всеми парами вершин можно использовать алгоритм Флойда–Уоршелла:

```
def FloydWarshall(d):  # d -- матрица начальных весов
    for k in range(V):
        for x in range(V):
            for y in range(V):
                d[x][y] = min(d[x][y], d[x][k] + d[k][y])
    return d
```

  Этот алгоритм имеет сложность $O(|V|^3)$ и потому слишком затратен для очень крупных сетей.

Альтернативой является алгоритм Джонсона, который позволяет эффективно получать all-pairs кратчайшие пути в разреженных графах.

2.7 Алгоритм Джонсона (1977)

Алгоритм Джонсона использует преобразование весов рёбер с помощью *потенциалов*. Пусть $\varphi : V \to \mathbb{R}$ — функция на вершинах (потенциал). Определим новую весовую функцию на рёбрах:

$$w_\varphi(x, y) = w(x, y) + \varphi(x) - \varphi(y).$$

Основные свойства:

1. Для любого пути $P(v, u)$ из v в u

$$w_\varphi(P(v, u)) = w(P(v, u)) + \varphi(v) - \varphi(u).$$

   То есть вес пути изменяется на константу, зависящую только от начальной и конечной вершины.

2. Кратчайшие пути по w и по w_φ совпадают как множества путей (меняются только численные значения их длины).

3. Веса циклов не меняются:

$$w_\varphi(C) = w(C)$$

   для любого цикла C.

Отсюда следует: если в исходном графе нет циклов отрицательного веса, то и в преобразованном графе их нет. При правильном выборе потенциала можно *избавиться* от отрицательных рёбер, сохранив все кратчайшие пути, и далее применять алгоритм Дейкстры.

Шаги алгоритма Джонсона.

1. Добавить фиктивную вершину $s^\star$ и соединить её нулевыми рёбрами со всеми вершинами исходного графа.

2. Запустить алгоритм Беллмана–Форда из вершины $s^\star$ и вычислить расстояния $\rho(s^\star, x)$ для всех $x \in V$. Если обнаружен отрицательный цикл, алгоритм прекращается: all-pairs кратчайшие пути не определены.

3. Определить потенциалы

$$\varphi(x) := \rho(s^\star, x), \qquad \forall x \in V,$$

   и новые веса рёбер

$$w_\varphi(v, u) := w(v, u) + \varphi(v) - \varphi(u), \qquad \forall (v, u) \in E.$$

4. В графе с весами w_φ запустить алгоритм Дейкстры из каждой вершины (или для нужных источников). Полученные расстояния $d_\varphi(v,u)$ далее переводятся обратно в оригинальные веса:
$$\rho(v,u) = d_\varphi(v,u) - \varphi(v) + \varphi(u).$$

Теорема 4 (Джонсон, 1977)**.** *Пусть в графе G отсутствуют циклы отрицательного веса и*
$$\varphi(x) = \rho(s^\star, x)$$
— расстояния от фиктивной вершины $s^\star$ до x, вычисленные алгоритмом Беллмана–Форда. Тогда для всех рёбер $(v,u) \in E$
$$w_\varphi(v,u) \geq 0.$$

Доказательство. По определению
$$w_\varphi(v,u) = w(v,u) + \varphi(v) - \varphi(u) = w(v,u) + \rho(s^\star, v) - \rho(s^\star, u).$$
Из определения кратчайших путей следует неравенство
$$\rho(s^\star, v) + w(v,u) \geq \rho(s^\star, u),$$
так как путь $s^\star \to v \to u$ не может быть короче кратчайшего пути $s^\star \to u$. Перенося $\rho(s^\star, u)$ вправо, получаем
$$w(v,u) + \rho(s^\star, v) - \rho(s^\star, u) \geq 0,$$
то есть $w_\varphi(v,u) \geq 0$. □

Сложность алгоритма Джонсона.

- Шаг с алгоритмом Беллмана–Форда имеет сложность $O(|V| \cdot |E|)$.
- Затем запускается алгоритм Дейкстры из каждой вершины. При использовании двоичной кучи сложность одного запуска равна $O(|E| \log |V|)$, так что суммарная сложность этапа Дейкстры составляет $O(|V| \cdot |E| \log |V|)$.

Итого общая сложность алгоритма Джонсона
$$O\big(|V| \cdot |E| + |V| \cdot |E| \log |V|\big) = O\big(|V| \cdot |E| \log |V|\big).$$
Для разреженных графов (где $|E| = O(|V|)$) это даёт оценку $O(|V|^2 \log |V|)$, а для плотных графов (где $|E| = O(|V|^2)$) — $O(|V|^3 \log |V|)$.

На практике для задач распределения потоков кратчайшие пути обычно ищутся многократным запуском алгоритма Дейкстры (по одному запуску на каждый исход), поскольку веса рёбер — времена/затраты — неотрицательны. Однако алгоритм Беллмана–Форда и

его вариации могут быть удобны для батч-обработки на GPU: основная операция — параллельная релаксация всех рёбер на каждом шаге. Выбор между BF и Дейкстрой зависит от постановки: для единичного источника и неотрицательных весов Дейкстра, как правило, быстрее, тогда как BF обеспечивает большую гибкость при массовых вычислениях или при наличии потенциальных отрицательных весов.

2.8 Алгоритм Франка–Вульфа.

В рамках сформулированной выше задачи Бекмана и праймал–дуальной схемы классический алгоритм Франка–Вульфа можно рассматривать как специализированный метод условного градиента в пространстве дуговых потоков. На каждой итерации k функционал $Z(x)$ линеаризуется в точке $x^{(k)}$, и решается вспомогательная линейная задача $\min_{y \in X} \langle \nabla Z(x^{(k)}), y \rangle$, где X — множество допустимых потоков. Для TAP градиент $\nabla Z(x)$ совпадает с вектором дуговых затрат $t(x)$, поэтому эта линейная подзадача эквивалентна *all-or-nothing* распределению по кратчайшим путям в сети с весами $t_a\big(x_a^{(k)}\big)$: для каждой O–D пары весь спрос направляется по кратчайшему пути, что даёт вспомогательный поток $y^{(k)}$. Далее строится новое приближение вдоль отрезка, соединяющего $x^{(k)}$ и $y^{(k)}$,

$$x^{(k+1)} = x^{(k)} + \alpha_k \big(y^{(k)} - x^{(k)}\big),$$

где шаг $\alpha_k \in (0, 1]$ определяется одномерным поиском минимума $Z\big(x^{(k)} + \alpha(y^{(k)} - x^{(k)})\big)$ или по стандартной формуле $\alpha_k = 2/(k+2)$. При непрерывных монотонно неубывающих функциях затрат $t_a(\cdot)$ функционал Бекмана выпукл, а при строгой монотонности — строго выпукл, что обеспечивает глобальную сходимость алгоритма Франка–Вульфа к единственному равновесному распределению по дугам. На практике этот метод удобен тем, что каждая итерация сводится к одной задаче поиска кратчайших путей и одной процедуре релаксации потоков вдоль направления $y^{(k)} - x^{(k)}$, а качество приближения можно контролировать через значение $Z(x^{(k)})$, UE-gap и duality gap. В численной части данной работы алгоритм Франка–Вульфа реализуется именно в таком виде и используется как «эталонный» градиентный метод при сравнении с альтернативными схемами на тестовой сети Sioux Falls.

2.9 Метод последовательных усреднений (MSA).

Метод последовательных усреднений (Method of Successive Averages, MSA) реализует ту же общую идею «all-or-nothing + релаксация», но использует заранее заданную убывающую последовательность шагов вместо точного линейного поиска. На шаге k при текущем потоке $x^{(k)}$ вычисляются дуговые затраты $t\big(x^{(k)}\big)$, после чего выполняется *all-or-nothing* распределение спроса по кратчайшим путям, что даёт вспомогательное решение $y^{(k)} \in X$. Новое приближение строится по формуле

$$x^{(k+1)} = x^{(k)} + \alpha_k \big(y^{(k)} - x^{(k)}\big),$$

где шаг

$$\alpha_k = \frac{1}{k+1}$$

выбирается заранее и не зависит от конкретного вида функций затрат. Такой выбор шагов удовлетворяет условиям $\sum_k \alpha_k = \infty$ и $\sum_k \alpha_k^2 < \infty$, типичным для схем последовательного усреднения, и обеспечивает сходимость $x^{(k)}$ к решению вариационного неравенства, эквивалентного пользовательскому равновесию, при непрерывных монотонных дуговых функциях $t_a(\cdot)$. Интуитивно MSA можно интерпретировать как взвешенное усреднение всех ранее полученных all-or-nothing распределений с постепенно уменьшающимся «весом» новых итераций. По сравнению с алгоритмом Франка–Вульфа метод MSA не требует вычислительно дорогого линейного поиска, обладает простой и устойчивой реализацией, но обычно сходится медленнее и чувствительнее к выбору критерия остановки. В настоящей работе MSA рассматривается как практическая, вычислительно простая альтернатива Франку–Вульфу, и в дальнейшем оба алгоритма сравниваются по скорости уменьшения UE-gap, поведению целевой функции Бекмана и времени работы на сети Sioux Falls.

2.10 Условия корректности и сходимости метода

В этом подразделе соберём воедино те предпосылки, которые обеспечивают корректность и сходимость реализуемых далее алгоритмов Франка–Вульфа и MSA. Часть структурных условий уже обсуждалась при формулировке задачи Бекмана и пользовательского равновесия (см. подраздел 2.2), поэтому здесь они лишь кратко напоминаются, а акцент делается на их связи с поведением алгоритмов.

- **Структурные свойства функций затрат.** Как было показано ранее, непрерывность дуговых функций $t_a(x_a)$ гарантирует существование решения UE, а их монотонная неубывающая форма (например, BPR-функции) обеспечивает выпуклость функции Бекмана и единственность равновесия по дуговым потокам. В контексте сходимости алгоритмов эти же свойства исключают наличие множества локальных минимумов и позволяют трактовать любую стационарную точку как глобальный минимум $Z(x)$.

- **Гладкость.** Дополнительная гладкость (дифференцируемость) функций $t_a(\cdot)$ делает функционал Бекмана $Z(x)$ непрерывно дифференцируемым и позволяет корректно использовать градиентную информацию. Для алгоритма Франка–Вульфа это означает, что направление условного градиента $\nabla Z(x^{(k)}) = t(x^{(k)})$ определено на каждой итерации, а линеаризация $Z(x)$ в окрестности $x^{(k)}$ адекватно отражает локальное поведение целевой функции. Негладкости (изломы, разрывы производных) не запрещают постановку UE, но могут ухудшать предсказуемость шага и замедлять сходимость.

- **Выбор шага и устойчивость итераций.** В праймал–дуальных методах ключевую роль играет выбор величины шага: α_k в обновлении потоков и, при явном обновлении

потенциалов, параметр ρ в дуальном шаге. Для метода последовательных усреднений классический выбор

$$\alpha_k = \frac{1}{k+1}$$

удовлетворяет условиям $\sum_k \alpha_k = \infty$ и $\sum_k \alpha_k^2 < \infty$ и тем самым обеспечивает сходимость $x^{(k)}$ к решению вариационного неравенства, эквивалентного UE. В алгоритме Франка–Вульфа шаг подбирается линейным поиском по $Z(x)$ либо по стандартной формуле типа $2/(k+2)$; в обоих случаях гарантируется монотонное уменьшение значения $Z(x)$. Слишком большие постоянные шаги (например, фиксированное α, не убывающее к нулю) могут приводить к осцилляциям вокруг равновесия, поэтому практически используют либо адаптивный линейный поиск, либо убывающие шаги.

- **Сглаживание (regularization).** Для ускорения сходимости и предотвращения цикличности на границе допустимой области иногда вводят дополнительные элементы сглаживания: небольшие регуляризационные добавки к функциям затрат (делающие их строго монотонными), усреднение потоков по нескольким последним итерациям, демпфирование изменений (обновление только части приращения потока, что эквивалентно выбору $\alpha_k < 1$ даже при «резком» направлении). Такие приёмы уменьшают эффект «переброса» потока между почти равноценными маршрутами и делают траекторию $\{x^{(k)}\}$ более плавной в евклидовой норме.

- **Теоретические гарантии.** При выполнении перечисленных условий (непрерывность, монотонность, выпуклость множества допустимых потоков и корректный выбор шагов) алгоритм Франка–Вульфа обладает доказанной сходимостью к минимуму функции Бекмана, а MSA — сходимостью в пределе к решению UE при выбранной схеме шагов. Аналогичные результаты известны и для других праймал–дуальных схем (например, метода Узавы / dual ascent), которые при подходящем выборе параметров сходятся к седловой точке лагранжиана и тем самым восстанавливают пользовательское равновесие.

Практические индикаторы того, что указанные теоретические предпосылки реализуются численно, обсуждались ранее в разделе 5: на каждой итерации оцениваются UE-gap, duality gap и нарушения ограничений баланса. При монотонных функциях затрат эти величины монотонно (или по крайней мере устойчиво) уменьшаются по мере работы алгоритма, что свидетельствует о приближении к оптимальному распределению. В случаях, когда реальные функции затрат выходят за рамки классических предположений (например, содержат немонотонные участки из-за специфики моделирования перегрузки), возможны несколько равновесий или «залипания» алгоритма, и тогда могут потребоваться более сложные эвристики или модифицированные схемы поиска, что уже выходит за рамки классического TAP UE.

3 Описание практической работы

Изложенная выше теоретическая часть задаёт строгую постановку задачи распределения транспортных потоков в форме задачи Бекмана, описывает условия существования и единственности пользовательского равновесия Уордропа, а также седловую формулировку через лагранжиан и соответствующие праймал–дуальные условия оптимальности. Кроме того, были рассмотрены алгоритмы поиска кратчайших путей (Беллмана–Форда, Дейкстры, Джонсона) и обсуждены критерии остановки и сходимости для итерационных методов решения TAP, включая алгоритм Франка–Вульфа и метод последовательных усреднений (MSA). Таким образом, сформирована математическая основа, позволяющая перейти к численной реализации и экспериментальному исследованию поведения конкретных алгоритмов на реальных сетях.

В дальнейшей практической части работы эти теоретические результаты используются для реализации и сравнения двух базовых методов решения статической задачи назначения потоков в состоянии пользовательского равновесия: алгоритма Франка–Вульфа и метода последовательных усреднений. В качестве тестового примера выбирается классическая сеть Sioux Falls с заданной O–D матрицей и дуговыми функциями затрат BPR-типа. Для обоих алгоритмов реализуется единый вычислительный конвейер: чтение и подготовка сетевых данных, многократный поиск кратчайших путей (all-or-nothing распределение), обновление дуговых потоков и оценка качества приближения к равновесию по показателям UE-gap, значению функции Бекмана и нарушениям ограничений баланса. Полученные реализации далее сравниваются по скорости сходимости, числу итераций и временам вычислений, что позволяет на практике оценить преимущества и недостатки каждого из методов применительно к задаче распределения транспортных потоков на сети Sioux Falls.

Вся проведенная практическая работа будет отражена в интерактивной среде Google Colab. Мы посчитали что удобнее всего продемонстрировать полученные результаты там.